

#1: Design any database with at least two entities and relationships between them. Draw suitable ER/EER diagram for the system

```
CREATE DATABASE college_db;  
USE college_db;
```

```
CREATE TABLE Student (  
    S_ID INT PRIMARY KEY,  
    S_Name VARCHAR(50),  
    Email VARCHAR(100),  
    Age INT  
);
```

```
CREATE TABLE Professor (  
    P_ID INT PRIMARY KEY,  
    P_Name VARCHAR(50),  
    Email VARCHAR(100),  
    Dept VARCHAR(50)  
);
```

```
CREATE TABLE Course (  
    C_ID INT PRIMARY KEY,  
    C_Name VARCHAR(50),  
    C_Prof INT,  
    FOREIGN KEY (C_Prof) REFERENCES Professor(P_ID)  
);
```

Enrollment Table (N:N → separate table)

```
CREATE TABLE Enrollment (  
    S_ID INT,  
    C_ID INT,  
    PRIMARY KEY (S_ID, C_ID),  
    FOREIGN KEY (S_ID) REFERENCES Student(S_ID),  
    FOREIGN KEY (C_ID) REFERENCES Course(C_ID)  
);
```

6. Insert Sample Data

```
INSERT INTO Student VALUES  
(1, 'Nilesh', 'nilesh@gmail.com', 20),  
(2, 'Rohit', 'rohit@gmail.com', 21);
```

```
INSERT INTO Professor VALUES  
(101, 'Dr. Sharma', 'sharma@gmail.com', 'IT'),  
(102, 'Dr. Patil', 'patil@gmail.com', 'CS');
```

```
INSERT INTO Course VALUES
(201, 'DBMS', 101),
(202, 'DSA', 102);
```

```
INSERT INTO Enrollment VALUES
(1, 201),
(1, 202),
(2, 201);
```

1. Student with Courses

```
SELECT s.S_Name, c.C_Name
FROM Student s
JOIN Enrollment e ON s.S_ID = e.S_ID
JOIN Course c ON e.C_ID = c.C_ID;
```

2. Course with Professor

```
SELECT c.C_Name, p.P_Name
FROM Course c
JOIN Professor p ON c.C_Prof = p.P_ID;
```

3. Full Data (Student + Course + Professor)

```
SELECT s.S_Name, c.C_Name, p.P_Name
FROM Student s
JOIN Enrollment e ON s.S_ID = e.S_ID
JOIN Course c ON e.C_ID = c.C_ID
JOIN Professor p ON c.C_Prof = p.P_ID;
```

Q.2.3.4. Design and implement a database using DDL and DML statements

CREATE → create table/database

ALTER → modify table (add/delete column)

TRUNCATE → delete all records (structure stays)

RENAME → rename table

DROP → delete table/database

```
CREATE DATABASE college_db;
```

```
CREATE TABLE student (
    id INT PRIMARY KEY,
    name VARCHAR(50)
);
```

```
ALTER TABLE Student ADD age INT;
INSERT INTO student VALUES( 1,'NILESH',20);
```

```
DESC student;  
ALTER TABLE Student DROP COLUMN age;  
DESC student;
```

```
TRUNCATE TABLE Student; //delete str
```

```
//rename it  
ALTER TABLE Student RENAME TO Students;
```

```
Drop :  
DROP TABLE Student;  
DROP DATABASE college_db;
```

Q.5.6.7.

Create Employee/student Table with primary key and foreign key constraints,
1.Alter table with add n modify b. Drop table

```
CREATE TABLE Department (  
    dept_id INT PRIMARY KEY,  
    dept_name VARCHAR(50)  
);  
CREATE TABLE Employee (  
    emp_id INT PRIMARY KEY,  
    emp_name VARCHAR(50),  
    salary DECIMAL(10,2),  
    dept_id INT,  
    FOREIGN KEY (dept_id) REFERENCES Department(dept_id)  
);
```

ADD column

```
ALTER TABLE Employee ADD email VARCHAR(100);
```

MODIFY column (MySQL syntax)

```
ALTER TABLE Employee MODIFY salary DECIMAL(12,2);
```

```
DROP  
DROP TABLE Employee;
```

Q.8. Create any database and implement all Alter queries

```
CREATE DATABASE company_db;
```

```
USE company_db;
```

```
CREATE TABLE Employee (  
    emp_id INT PRIMARY KEY,  
    emp_name VARCHAR(50),  
    salary DECIMAL(10,2)
```

```
);
```

Insert something

Add column:

```
ALTER TABLE Employee ADD email VARCHAR(100);
```

modify

```
ALTER TABLE Employee MODIFY salary DECIMAL(12,2);
```

Rename column

```
ALTER TABLE Employee RENAME COLUMN emp_name TO name;
```

Rename table

```
ALTER TABLE Employee RENAME TO Employees;
```

Drop column:

```
alter table employees drop column name;
```

Add foreign key:

```
CREATE TABLE Department (  
    dept_id INT PRIMARY KEY  
);
```

```
ALTER TABLE Employee ADD dept_id INT;
```

```
ALTER TABLE Employee
```

```
ADD FOREIGN KEY (dept_id) REFERENCES Department(dept_id);
```

Q.9 Implementation of relational operators in SQL.

```
CREATE TABLE Employee (  
    emp_id INT,  
    name VARCHAR(50),
```

```
    salary INT,  
    age INT  
);  
INSERT INTO Employee VALUES  
(1, 'Nilesh', 50000, 20),  
(2, 'Rohit', 60000, 22),  
(3, 'Amit', 40000, 19);
```

```
SELECT * FROM Employee WHERE salary = 50000;  
SELECT * FROM Employee WHERE salary != 50000;  
SELECT * FROM Employee WHERE salary >50000;  
SELECT * FROM Employee WHERE salary <50000;
```

Q.10 pattern matching

```
CREATE TABLE Student (  
    id INT,  
    name VARCHAR(50)  
);
```

```
INSERT INTO Student VALUES  
(1, 'Nilesh'),  
(2, 'Rohit'),  
(3, 'Amit'),  
(4, 'Neha');
```

Implementation

➤ Starts with 'N'

```
SELECT * FROM Student WHERE name LIKE 'N%';
```

➤ Ends with 't'

```
SELECT * FROM Student WHERE name LIKE '%t';
```

➤ Contains 'it'

```
SELECT * FROM Student WHERE name LIKE '%it%';
```

➤ Exactly 5 characters

```
SELECT * FROM Student WHERE name LIKE '_____';
```

➤ Starts with 'A' and 4 letters total

```
SELECT * FROM Student WHERE name LIKE 'A_____';
```

Q.11 Implementation of Aggregate functions with group by and having clauses.

```
CREATE TABLE Employee (  
    emp_id INT,  
    name VARCHAR(50),  
    dept VARCHAR(50),  
    salary INT  
);
```

```
INSERT INTO Employee VALUES  
(1, 'Nilesh', 'IT', 50000),  
(2, 'Rohit', 'IT', 60000),  
(3, 'Amit', 'HR', 40000),  
(4, 'Neha', 'HR', 45000),  
(5, 'Pooja', 'IT', 70000);
```

GROUP BY

👉 Groups rows based on column

```
SELECT dept, COUNT(*)  
FROM Employee  
GROUP BY dept;
```

Aggregate with GROUP BY

```
SELECT dept, SUM(salary), AVG(salary), MAX(salary), MIN(salary)  
FROM Employee  
GROUP BY dept;
```

HAVING Clause

👉 Used to filter **groups** (not rows)

```
SELECT dept, SUM(salary)
FROM Employee
GROUP BY dept
HAVING SUM(salary) > 100000;
```

Q.12 .Implementation of Nested queries.

```
CREATE TABLE Employee (
    emp_id INT,
    name VARCHAR(50),
    dept VARCHAR(50),
    salary INT
);
```

```
INSERT INTO Employee VALUES
(1, 'Nilesh', 'IT', 50000),
(2, 'Rohit', 'IT', 60000),
(3, 'Amit', 'HR', 40000),
(4, 'Neha', 'HR', 45000);
```

```
SELECT name, salary
FROM Employee
WHERE salary > (SELECT AVG(salary) FROM Employee);
```

2. Multiple Row Subquery (IN)

👉 Employees from same dept as 'Nilesh'

```
SELECT name
FROM Employee
WHERE dept IN (
    SELECT dept FROM Employee WHERE name = 'Nilesh'
);
```

Not in

```
SELECT name
FROM employee
WHERE dept NOT IN (
    SELECT dept FROM employee WHERE dept = 'IT'
);
```

5. Correlated Subquery

👉 Employees earning more than avg of their dept

```
SELECT name, dept, salary
FROM Employee e1
WHERE salary > (
    SELECT AVG(salary)
    FROM Employee e2
    WHERE e1.dept = e2.dept
);
```

Q.13, implementation of Index Operation

```
CREATE TABLE Employee (
    emp_id INT,
    name VARCHAR(50),
    salary INT
);
```

1. Create Index

```
CREATE INDEX idx_name ON Employee(name);
```

👉 Index on **name** column

2. Create Unique Index

```
CREATE UNIQUE INDEX idx_empid ON Employee(emp_id);
```

👉 No duplicate values allowed

3. Create Composite Index (multiple columns)

```
CREATE INDEX idx_name_salary ON Employee(name, salary);
```

4. View Index

👉 MySQL:

```
SHOW INDEX FROM Employee;
```

14.implementaion of set operators

Set Operators in SQL — Implementation (to the point)

👉 Used to **combine results of multiple SELECT queries**

Types

- **UNION** → combine + remove duplicates
 - **UNION ALL** → combine + keep duplicates
 - **INTERSECT** → common rows
-

Example Tables

```
CREATE TABLE A (  
  id INT  
);
```

```
CREATE TABLE B (  
  id INT  
);
```

```
INSERT INTO A VALUES (1), (2), (3);  
INSERT INTO B VALUES (2), (3), (4);
```

1. UNION

```
SELECT id FROM A  
UNION  
SELECT id FROM B;
```

👉 Output: 1, 2, 3, 4 (no duplicates)

2. UNION ALL

```
SELECT id FROM A  
UNION ALL  
SELECT id FROM B;
```

👉 Output: 1, 2, 3, 2, 3, 4 (duplicates allowed)

3. INTERSECT (common)

```
SELECT id FROM A  
INTERSECT  
SELECT id FROM B;
```

👉 Output: 2, 3

15.implementation of view

✓ Views in SQL — Implementation (to the point)

👉 View = virtual table (based on SELECT query)



Example Table

```
CREATE TABLE Employee (  
    emp_id INT,  
    name VARCHAR(50),  
    dept VARCHAR(50),  
    salary INT  
);
```

```
INSERT INTO Employee VALUES  
(1, 'Nilesh', 'IT', 50000),  
(2, 'Rohit', 'IT', 60000),  
(3, 'Amit', 'HR', 40000);
```



1. Create View

```
CREATE VIEW IT_Employees AS  
SELECT emp_id, name, salary  
FROM Employee  
WHERE dept = 'IT';
```



2. Use View

```
SELECT * FROM IT_Employees;
```



3. Update View

👉 MySQL:

```
CREATE OR REPLACE VIEW IT_Employees AS
SELECT emp_id, name
FROM Employee
WHERE dept = 'IT';
```

16.Sorting in SQL



Sorting in SQL — Implementation (to the point)

👉 Done using **ORDER BY**



Example Table

```
CREATE TABLE Employee (
  emp_id INT,
  name VARCHAR(50),
  salary INT,
  age INT
);

INSERT INTO Employee VALUES
(1, 'Nilesh', 50000, 20),
(2, 'Rohit', 60000, 22),
(3, 'Amit', 40000, 19);
```



1. Ascending Order (default)

```
SELECT * FROM Employee  
ORDER BY salary ASC;
```

👉 Small → Large

2. Descending Order

```
SELECT * FROM Employee  
ORDER BY salary DESC;
```

👉 Large → Small

3. Sort by Multiple Columns

```
SELECT * FROM Employee  
ORDER BY salary DESC, age ASC;
```

4. Sort by Column Position

```
SELECT * FROM Employee  
ORDER BY 3 DESC;
```

👉 3rd column = salary

5. Sort by Name

```
SELECT * FROM Employee
```

ORDER BY name;

17.procedure t

```
CREATE TABLE Employee (  
    emp_id INT PRIMARY KEY,  
    name VARCHAR(50),  
    dept VARCHAR(50),  
    salary INT  
);
```

```
INSERT INTO Employee VALUES  
(1, 'Nilesh', 'IT', 50000),  
(2, 'Rohit', 'IT', 60000),  
(3, 'Amit', 'HR', 40000);
```

17 SQL Stored Procedure

✓ SQL Stored Procedure — Implementation (to the point)

👉 Task: Get employees of a specific department

1. Create Table

```
CREATE TABLE Employee (  
    emp_id INT PRIMARY KEY,  
    name VARCHAR(50),  
    dept VARCHAR(50),  
    salary INT  
);
```

```
INSERT INTO Employee VALUES  
(1, 'Nilesh', 'IT', 50000),  
(2, 'Rohit', 'IT', 60000),  
(3, 'Amit', 'HR', 40000);
```



2. Create Stored Procedure (MySQL)

```
DELIMITER $$
```

```
CREATE PROCEDURE GetEmployeesByDept(  
    IN dept_name VARCHAR(50)  
)  
BEGIN  
    SELECT *  
    FROM Employee  
    WHERE dept = dept_name;  
END $$
```

```
DELIMITER ;
```



3. Execute Procedure

```
CALL GetEmployeesByDept('IT');
```

18 sql FUNCTION



1. Create Table

```
CREATE TABLE Employee (  
    emp_id INT PRIMARY KEY,  
    name VARCHAR(50),  
    dept VARCHAR(50),  
    salary INT  
);
```

```
INSERT INTO Employee VALUES  
(1, 'Nilesh', 'IT', 50000),  
(2, 'Rohit', 'IT', 60000),  
(3, 'Amit', 'HR', 40000);
```



2. Create Stored Procedure (MySQL)

```
DELIMITER $$
```

```
CREATE PROCEDURE GetEmployeesByDept(  
    IN dept_name VARCHAR(50)  
)  
BEGIN  
    SELECT *  
    FROM Employee  
    WHERE dept = dept_name;  
END $$
```

```
DELIMITER ;
```



3. Execute Procedure

```
CALL GetEmployeesByDept('IT');
```

19. TRIGGERS



Database Triggers — Implementation (to the point)

👉 **Trigger** = auto-executes when event occurs (INSERT / UPDATE / DELETE)



1. Create Tables

```
CREATE TABLE Employee (  
    emp_id INT PRIMARY KEY,  
    name VARCHAR(50),  
    salary INT  
);
```

```
CREATE TABLE Audit (  
    emp_id INT,  
    action VARCHAR(20),  
    action_time TIMESTAMP
```


);

2. Create Trigger

👉 Task: **Log insert operation into Audit table**

MySQL

DELIMITER \$\$

```
CREATE TRIGGER after_employee_insert
AFTER INSERT ON Employee
FOR EACH ROW
BEGIN
    INSERT INTO Audit(emp_id, action, action_time)
    VALUES (NEW.emp_id, 'INSERT', NOW());
END $$
```

DELIMITER ;

3. Execute (Trigger fires automatically)

```
INSERT INTO Employee VALUES (1, 'Nilesh', 50000);
```

4. Verify

```
SELECT * FROM Audit;
```

👉 Output:

```
1  INSERT  2026-05-06 ...
```

20. JOIN



Join in SQL — Implementation (to the point)



JOIN = combine data from multiple tables



Example Tables

```
CREATE TABLE Department (  
    dept_id INT PRIMARY KEY,  
    dept_name VARCHAR(50)  
);
```

```
CREATE TABLE Employee (  
    emp_id INT,  
    name VARCHAR(50),  
    dept_id INT  
);
```

```
INSERT INTO Department VALUES  
(1, 'IT'),  
(2, 'HR');
```

```
INSERT INTO Employee VALUES  
(1, 'Nilesh', 1),  
(2, 'Rohit', 1),  
(3, 'Amit', 2),  
(4, 'Neha', NULL);
```

1. INNER JOIN

👉 Only matching records

```
SELECT e.name, d.dept_name  
FROM Employee e  
INNER JOIN Department d  
ON e.dept_id = d.dept_id;
```

2. LEFT JOIN

👉 All from left + matched from right

```
SELECT e.name, d.dept_name  
FROM Employee e  
LEFT JOIN Department d  
ON e.dept_id = d.dept_id;
```

3. RIGHT JOIN

👉 All from right + matched from left

```
SELECT e.name, d.dept_name  
FROM Employee e  
RIGHT JOIN Department d  
ON e.dept_id = d.dept_id;
```

4. FULL JOIN (PostgreSQL)

👉 All records from both tables

```
SELECT e.name, d.dept_name  
FROM Employee e  
FULL JOIN Department d  
ON e.dept_id = d.dept_id;
```

5. CROSS JOIN

👉 Cartesian product

```
SELECT e.name, d.dept_name  
FROM Employee e  
CROSS JOIN Department d;
```
